

Chainlink事件合约报价BTC ETH集成方案[INTERNAL]

面向对象：TurboFlow 内部工程团队

目标：将 Chainlink BTC/ETH 价格接入当前猜涨跌 / event-contract 产品，Chainlink 作为 Primary price feed，现有 TurboFlow Oracle 价格源作为 Backup。

1. 项目目标

当前猜涨跌产品依赖 `cex-oracle-service` 推送的 BTC/ETH 市场价格完成开仓价锁定、结算价格读取、风控校验和 stale price 拦截。本项目目标是在不改变下游订单 / 结算 payload 的前提下，将 Chainlink Data Streams 接入为 BTC/ETH 的主价格源。

目标效果：

- BTC/ETH 价格优先使用 Chainlink Data Streams。
- Chainlink 不可用、超时、解码失败、价格异常或偏离现有价格源过大时，自动回退到现有 TurboFlow Oracle 聚合价格。
- `surfv2-dex-svm-keeper` 继续消费现有 `/ws/price`，尽量不改 keeper 和合约结算路径。
- 保留观测、告警和灰度开关，支持 Shadow -> UAT -> Production 分阶段上线。

2. 技术方案简述

推荐接入 Chainlink Data Streams，而不是第一阶段直接接 EVM AggregatorV3Interface。

原因：

- P1 任务里已有 Data Streams testnet/mainnet credentials，且当前技术栈偏 Go。
- Data Streams 提供低延迟 offchain 报告，支持 WebSocket / REST / SDK 接入，并可在未来扩展到 onchain verification。

- 现有 TurboFlow 价格分发链路是 offchain oracle service -> keeper, Data Streams 更适合以 cex-oracle-service adapter 方式接入。

目标架构:

Chainlink Data Streams

```
-> cex-oracle-service/exchanges/chainlinkstreams
-> datafeed.OnReceiveTicker(MarketData{Origin:
"chainlinkDataStreams"})
-> Chainlink primary / existing oracle backup selector
-> existing PriceData over /ws/price
-> surfv2-dex-svm-keeper
-> prediction open / settlement
```

3. Chainlink 技术规格

使用 SDK :

github.com/smartcontractkit/data-streams-sdk/go

建议配置项:

```
CHAINLINK_STREAMS_API_KEY
CHAINLINK_STREAMS_API_SECRET
CHAINLINK_STREAMS_REST_URL
CHAINLINK_STREAMS_WS_URL
CHAINLINK_STREAMS_WS_HA=true
```

建议业务配置:

```
{
  "BTCUSDT": {
    "feed_id": "<full Chainlink BTC/USDT feed id>",
    "max_staleness_ms": 2000,
    "max_deviation_from_backup": "0.01"
  },
  "ETHUSDT": {
    "feed_id": "<full Chainlink ETH/USDT feed id>",
    "max_staleness_ms": 2000,
    "max_deviation_from_backup": "0.01"
  }
}
```

Feed 选择:

- 优先使用 Chainlink BTC / USDT 和 ETH / USDT streams, 对应当前交易对 BTCUSDT / ETHUSDT。
- 上线前必须通过 Chainlink dashboard 或 SDK GetFeeds(ctx) 校验账号可见的完整 feed ID。
- 如果账号权限中没有 USDT streams, 再评估 BTC / USD、ETH / USD + USDT / USD 的换算方案。该方案多一层换算风险, 不作为首选。

报告字段使用建议:

- 发布价格: BenchmarkPrice
- 监控字段: Bid、Ask、report timestamp、feed ID
- 时间戳: 优先使用 report observation / valid timestamp, 转换成 Unix seconds 写入 MarketData.Time
- 拒绝条件: feed ID 不匹配、decode 失败、benchmark price <= 0、报告超时、价格偏离 backup 超阈值

4. 当前价格链路调查结论

当前链路:

exchange adapters

- > datafeed.MarketData
- > cex-oracle-service aggregation
- > base/entity.PriceData
- > /ws/price
- > surfv2-dex-svm-keeper OrderPriceProcessor
- > model.GlobalCfgMgr.SetPairTicker
- > prediction open / settlement

关键文件:

- cex-oracle-service/datafeed/models/marketdata.go
 - MarketData 是内部 source tick 格式, 包含 Origin、Token、Price、Time、PairId、OracleKey、OriginType。
- cex-oracle-service/datafeed/server.go
 - SubscribePrice 根据 pair price rules 订阅各报价源。
 - OnReceiveTicker 将报价源 tick 路由到价格处理队列。
 - procTickerRoutine 写入 source price、计算最终价格并通过 Cast.Write(pData) 推送。
- cex-oracle-service/datafeed/service_check.go
 - CalcPairPriceByWeight 计算现有聚合价格。

- 当前实现里 `weight := 1.0`，不是严格的 primary / backup 优先级模型。
- `base/entity/marketdata.go`
 - `PriceData` 是 `/ws/price` 的下游 payload。
- `surfv2-dex-svm-keeper/domain/services/schedule_service.go`
 - keeper 启动时订阅 oracle `/ws/price`，并从 `/trade/pair/price/list` 预加载价格。
- `surfv2-dex-svm-keeper/domain/services/order_price_processor.go`
 - `ReceivePriceData` 接收 tick，`procPairOrderTrigger` 写入 `GlobalCfgMgr.SetPairTicker`。
- `surfv2-dex-svm-keeper/domain/services/api_order_service.go`
 - 猜涨跌开仓目前有 1 秒 stale price 拦截。
- `surfv2-dex-svm-keeper/domain/services/predict_market_service.go`
 - 猜涨跌结算使用 `GlobalCfgMgr.GetPairPriceTimeout(order.PairID)`。

重要结论：只把 `chainlinkDataStreams` 加进 `oracle_rule_token.price_rule` 不足以实现“Chainlink primary、现有源 backup”。需要在 oracle aggregation 层增加显式 primary/fallback selector。

5. 技术交接方案

5.1 新增 Chainlink adapter

Repo : `cex-oracle-service`

建议新增：

`exchanges/chainlinkstreams/processor.go`
`exchanges/chainlinkstreams/config.go`

需要修改：

`constants/constants.go`
`datafeed/server.go`

新增 source constant：

```
const ChainlinkDataStreams = "chainlinkDataStreams"
```

Adapter 需要实现 `datafeed/models.Exchange`：

```
type Exchange interface {
    GetSubLen() int
}
```

```

    AddSubscribe(TokenChange)
    Subscribe(subscribeType constants.SubscribeType) AdapterFunc
    Sync() error
    IsSymbolOnline(symbol string) bool
    GetSymbolPrice(symbol string) (float64, bool)
    GetAllSymbolPrices() map[string]decimal.Decimal
    SaveSymbols()
    IsRpcHealthy() bool
}

```

5.2 接入 ExchangeFactory

修改点：

- ExchangeFactory 增加 ChainlinkDataStreamsExchange
models.Exchange
- InitExchanges 初始化 chainlinkstreams.NewProcessService(svr)
- GetExchange 支持 constants.ChainlinkDataStreams
- GetSubscribeMap 增加 m[constants.ChainlinkDataStreams] =
svr.ChainlinkDataStreamsExchange.Subscribe(subscribeType)
- SubscribePrice 的 source switch 增加 constants.ChainlinkDataStreams
- OnReceiveTicker 的 CEX-like source switch 增加
constants.ChainlinkDataStreams

5.3 实现 primary/fallback selector

建议不要直接改坏现有 CalcPairPriceByWeight 行为。更稳妥的方式：

1. 保留现有聚合逻辑作为 backup。
2. 增加一个选择层，例如：

```

func (svr *Server) SelectFinalPrice(token *entities.OracleToken)
    (price decimal.Decimal, source string, ok bool)

```

1. 选择逻辑：

if token 是 BTC/ETH 且 Chainlink primary 开关开启：

```

    if Chainlink source price fresh && price > 0 && deviation <=
threshold:
    return Chainlink price
else:
    return existing CalcPairPriceByWeight(token)

```

```
else:
    return existing CalcPairPriceByWeight(token)
```

1. procTickerRoutine 发布 pData.Price 时调用 selector。

偏离保护建议：

```
deviation = abs(chainlink_price - backup_price) / backup_price
default threshold = 1%
```

第一版策略：

- Chainlink 和 backup 都可用，且偏离超过阈值：回退 backup，并发 monitor alert。
- Chainlink 不可用：回退 backup。
- Chainlink 可用但 backup 不可用：允许 Chainlink，但记录 backup_unavailable。
- 两者都不可用：不发布价格，让 keeper stale price 保护生效。

5.4 DB / Runtime config

至少需要：

- quote_source 增加 chainlinkDataStreams。
- BTC/ETH 对应 oracle_rule_token.price_rule 增加 chainlinkDataStreams，用于触发订阅。
- 增加 Chainlink primary mapping config：oracle_key / pair symbol -> feed ID、staleness threshold、deviation threshold、shadow/primary mode。

配置模式建议：

```
{
  "enabled": true,
  "mode": "shadow",
  "pairs": {
    "BTCUSDT": {
      "feed_id": "<full feed id>",
      "max_staleness_ms": 2000,
      "max_deviation_from_backup": "0.01"
    },
    "ETHUSDT": {
      "feed_id": "<full feed id>",
      "max_staleness_ms": 2000,
      "max_deviation_from_backup": "0.01"
    }
  }
}
```

```
}  
}
```

mode 建议：

- off: 不订阅或不使用 Chainlink。
- shadow: 订阅并记录 Chainlink vs backup，但最终价格仍用现有聚合。
- primary: Chainlink 作为 primary，异常时 fallback backup。

6. 测试计划

Unit tests：

- Chainlink report decode 成功，输出 MarketData。
- report feed ID 不匹配时拒绝。
- benchmark price ≤ 0 时拒绝。
- report stale 时拒绝。
- Chainlink fresh 时 selector 返回 Chainlink。
- Chainlink stale 时 selector 返回 backup。
- Chainlink 和 backup 偏离超过阈值时返回 backup，并记录 fallback reason。
- 非 BTC/ETH pair 保持现有聚合逻辑不变。

Integration tests：

- mock Chainlink stream + mock existing source，验证 /ws/price payload schema 不变。
- 验证 /trade/pair/price/list 返回 selector 后的最终价格。
- keeper 订阅 /ws/price 后，GlobalCfgMgr.GetPairPriceTimeout 能读到新价格。
- Chainlink 断开、decode error、stale report 时，价格自动 fallback 到现有源。

UAT：

- Shadow 模式至少覆盖一个高波动窗口和一个低波动窗口。
- 对比 BTC/ETH Chainlink price、backup price、final price、fallback rate。
- 验证凭证错误、WebSocket 断开、feed ID 错误、report stale 的恢复路径。
- 验证猜涨跌开仓不会因正常 Chainlink tick 延迟触发 stale price 拒单。

7. 上线步骤

1. SIT：接入 SDK、mock 测试、shadow mode。

2. SIT：用真实 Chainlink testnet/mainnet credentials 校验 GetFeeds(ctx) 和 stream read。
3. UAT：BTC shadow -> BTC primary -> ETH shadow -> ETH primary。
4. Production：先只开 shadow，观察 24 小时。
5. Production：BTC primary 小流量窗口上线。
6. Production：ETH primary 上线。
7. 保留紧急回滚开关：mode=shadow 或 mode=off。

8. 监控与告警（可以结合现有的报警机制一起考虑）

建议新增：

- chainlink_stream_connected{feed_id,pair}
- chainlink_report_age_ms{feed_id,pair}
- chainlink_decode_errors_total{feed_id,pair}
- chainlink_fallback_total{pair,reason}
- chainlink_backup_deviation{pair}
- oracle_final_price_source{pair,source}

告警建议：

- BTC/ETH Chainlink report 超过 2 秒未更新。
- fallback rate 持续高于阈值。
- Chainlink vs backup deviation 超过配置阈值。
- SDK HA mode 连接数不足或重连频繁。
- /ws/price 对 keeper 推送延迟异常。

9. 风险与待确认项

- 当前猜涨跌开仓硬编码 1 秒 stale price threshold，Chainlink max_staleness_ms 是否需要低于 1 秒，需要风险和产品确认。
- 结算时如果 Chainlink 异常，是否总是 fallback backup，还是某些情况应暂停结算，需要风险确认。
- 是否需要 event contract 未来进行 onchain report verification。P1 建议不做，但接口设计要预留。
- Chainlink 账号权限、feed entitlement、API key rotation、生产 secret owner 需要明确。
- 任务原始文档里出现了 credentials，建议迁移到 secret 管理并轮换。

10. 参考资料

- Chainlink Data Streams Overview: <https://docs.chain.link/data-streams>
- Chainlink Data Streams API Reference: <https://docs.chain.link/data-streams/reference/data-streams-api>
- Chainlink Data Streams Dashboard: <https://data.chain.link/streams>
- Chainlink Data Streams SDK: <https://github.com/smartcontractkit/data-streams-sdk>